



《AI大模型Ragent项目》——意图识别与多路由调度策略

来自： 拿个offer-开源&项目实战



马丁

2026年03月08日 19:41

上一篇讲了 Query 改写，RAG 系统的多轮对话能力已经比较完整了：会话记忆让模型记住了上下文，Query 改写让检索系统也能理解上下文，检索效果大幅提升。

但你把这套系统上线之后，会发现一个尴尬的问题。

用户说“你好，我想咨询一下”，系统拿“你好我想咨询一下”去向量数据库里检索，召回了退货政策、保修条款、配送时效三个 chunk，然后模型基于这些 chunk 回答：“您好，关于退货政策，我们提供七天无理由退货服务……”——用户只是打了个招呼，你给人家讲退货政策？

用户说“我查一下我的年假余额”，系统去知识库里检索年假余额，召回了公司年假制度：正式员工每年享有 10 天带薪年假……，模型回答：“根据公司年假制度，正式员工每年享有 10 天带薪年假。”——用户想知道自己还剩几天年假，不是问年假制度。这个问题应该调 HR 系统的 API，不是查知识库。

问题出在哪？你的 RAG 系统是一根筋的——不管用户说什么，都往知识库里检索。就像一家医院没有分诊台，所有患者来了都直接送去内科，牙疼的、骨折的、做体检的全挤在内科门口。

今天要讲的就是给 RAG 系统装一个分诊台——**意图识别与问题路由**。用户消息进来，先判断它应该走哪条路：查知识库、调工具、直接闲聊、还是反问用户补充信息，然后把请求分发到对应的处理流程。

为什么需要意图识别

1. 一根筋的 RAG 系统会出什么问题

不做意图识别，所有消息都走 RAG 检索，会遇到三类问题：

用户消息类型	走 RAG 检索的结果	实际应该怎么处理
闲聊（你好、谢谢、哈哈）	召回不相关的 chunk，模型基于不相关内容强行回答，比不检索还差	模型直接回答，不需要检索
工具调用（查我的订单、我还剩几天年假）	知识库里没有用户的个人数据，检索不到有用信息，模型说“找不到相关信息”	调 Function Call / MCP 查实时数据
模糊问题（有什么推荐的吗、怎么办）	检索结果分散在多个主题，模型乱答一通	反问用户补充信息，明确意图后再处理

闲聊走检索，比不检索还差 ——这一点很多人没意识到。RAG 系统的生成策略里通常有一条规则：“基于以下参考资料回答用户问题”。如果检索到的 chunk 和用户问题完全不相关，模型要么硬着头皮基于不相关的内容回答（答案很奇怪），要么触发兜底回复“抱歉，找不到相关信息”（用户只是说了句你好，你说找不到相关信息？）。

如果不走检索，模型直接回答“你好”，反而更自然。

2. 医院分诊台的类比

意图识别的作用，就像医院的分诊台。

患者来了不是直接看病，先到分诊台：护士问你哪里不舒服，然后告诉你该去哪个科室。牙疼去口腔科，骨折去骨科，做体检去体检中心，不确定什么病的先去全科看看。

RAG 系统也一样。用户消息进来，先经过意图识别这个分诊台：

- 问产品信息、政策规定 → 去“知识库科室”（RAG 检索）
- 查订单、查年假、提交退货 → 去“工具科室”（Function Call / MCP）
- 打招呼、说谢谢 → 去“前台”（直接回复）
- 说不清楚想要什么 → “请您再描述一下症状”（引导澄清）

没有分诊台的医院效率低、体验差；没有意图识别的 RAG 系统也是一样。

四种意图类型

在电商客服场景下，用户消息可以分为四种意图类型。这个分类体系不是固定的——不同业务场景可能有不同的分类方式，但这四类覆盖了大多数场景。

1. 知识检索（Knowledge Retrieval）

用户在问一个知识库能回答的问题。这类问题的特点是：答案存在于你的知识库文档里，是“静态”的、通用的知识。

典型问题：

iPhone 16 Pro 的退货政策是什么？
保修期内屏幕碎了能免费换吗？
你们的配送范围覆盖哪些城市？
会员等级有什么权益？

处理方式：走完整的 RAG 检索流程——Query 改写 → 向量检索 → 重排序 → 生成答案。

2. 工具调用 (Tool Invocation)

用户想查询个人数据、实时信息，或者执行某个操作。这类问题的特点是：答案不在知识库里，需要调用外部系统的 API 才能获取。
典型问题：

帮我查一下订单 #12345 的物流状态（查个人订单数据）
我还剩几天年假？（查 HR 系统）
帮我申请退货（执行操作）
今天有什么优惠活动？（查实时数据）

处理方式：走 Function Call / MCP 流程——识别出需要调用哪个工具，传入参数，执行工具，把结果返回给模型生成最终答案。

怎么区分知识检索和工具调用？关键看答案的来源。如果答案在你的知识库文档里（退货政策、保修条款、产品参数），就是知识检索；如果答案需要查数据库、调 API、访问外部系统（某个订单的状态、某个人的年假余额、实时库存），就是工具调用。

3. 闲聊对话 (Chitchat)

用户在闲聊、打招呼、表达感谢、开玩笑，不涉及具体的业务问题。
典型问题：

你好
谢谢你的帮助
你是 AI 还是真人？
今天心情不好
哈哈，你真有趣

处理方式：模型直接回答，不走检索，也不调工具。System Prompt 里定义好闲聊时的回复风格就行。
前面说过，闲聊消息走 RAG 检索反而更差。模型自己就能很好地处理这类对话，不需要知识库的辅助。

4. 引导澄清 (Clarification)

用户的问题太模糊，系统无法确定该走哪条路。这时候不应该硬答，而是反问用户补充信息。
典型问题：

有什么推荐的吗？（推荐什么？手机？配件？）
怎么办？（什么怎么办？退货？维修？）
帮我看看（看什么？订单？产品？）
出问题了（什么产品？什么问题？）

处理方式：不检索也不调工具，直接返回一个引导性的问题，让用户补充关键信息。

引导澄清不是万能的，不能动不动就反问用户。如果用户的问题虽然简短但意图明确（比如“价格呢”，在聊 iPhone 16 Pro 的上下文中意图很清楚），就不应该反问，而是结合对话历史直接处理。只有在真的无法判断意图时才引导澄清。

5. 四种意图对比

意图类型	触发条件	处理流程	典型示例
知识检索	问知识库里有的通用知识	Query 改写 → 检索 → 生成答案	退货政策是什么
工具调用	查个人数据 / 实时信息 / 执行操作	Function Call / MCP	查我的订单状态
闲聊对话	打招呼 / 感谢 / 闲聊	模型直接回答	你好、谢谢
引导澄清	问题模糊、信息不足	反问用户补充信息	有什么推荐的

实际业务中，意图分类体系可能更细。比如电商场景可能拆出“投诉”“催单”“比价”等专门的意图类别。但核心思路不变：先分类，再路由。从这四种基础类型开始，根据业务需要逐步细化。

意图识别的三种方案

知道了要分哪几类意图，接下来的问题是：怎么判断一条用户消息属于哪个类别？

1. 规则匹配方案

最简单直接的方案：用关键词和正则表达式判断意图。

```
public class RuleBasedClassifier {

    /** 闲聊关键词 */
    private static final Set<String> CHITCHAT_KEYWORDS = Set.of(
        "你好", "您好", "谢谢", "感谢", "再见", "拜拜",
        "哈哈", "嗯嗯", "好的", "收到", "明白了"
    );

    /** 工具调用关键词 */
    private static final Set<String> TOOL_KEYWORDS = Set.of(
        "我的订单", "查订单", "订单号", "物流状态",
        "年假", "余额", "申请退货", "提交",
        "修改地址", "取消订单"
    );

    public String classify(String query) {
        // 完全匹配闲聊关键词（短消息）
        if (query.length() <= 6 && CHITCHAT_KEYWORDS.contains(query)) {
            return "chitchat";
        }

        // 包含工具调用关键词
        for (String keyword : TOOL_KEYWORDS) {
            if (query.contains(keyword)) {
                return "tool";
            }
        }

        // 消息太短且不是闲聊，可能需要澄清
        if (query.length() < 5) {
            return "clarification";
        }

        // 默认走知识检索
        return "knowledge";
    }
}
```

规则匹配的优缺点很明显：

维度	表现
速度	极快（微秒级）
准确率	低——只能匹配写死的关键词，覆盖不了用户的各种表达方式
成本	零（不调模型）
维护成本	高——每发现一个 bad case 就要加一条规则，越加越多越加越乱

规则方案最大的问题是**无法理解语义**。“帮我看剩几天假”没有命中“年假”关键词，就会被错分为知识检索。“能退吗”——是问退货政策（知识检索）还是想发起退货（工具调用）？规则无法区分。

规则方案适合做**第一层快速过滤** —— 把最明确的意图快速识别出来（“你好”就是闲聊，“查订单 #12345”就是工具调用），不确定的交给大模型处理。

2. 大模型分类方案

用大模型做意图分类，核心是设计好分类 Prompt。

2.1 分类 Prompt 的设计

你是一个意图分类助手。根据对话历史和用户的最新消息，判断用户的意图类别。

意图类别定义：

1. **knowledge** - 知识检索：用户在询问产品信息、政策规定、操作指南等知识库中的内容。

示例：“iPhone 16 Pro 的退货政策是什么”“保修期多久”“配送范围覆盖哪些城市”

2. **tool** - 工具调用：用户想查询个人数据、实时信息，或执行某个操作。

示例：“查一下我的订单状态”“帮我申请退货”“我还剩几天年假”

3. **chitchat** - 闲聊对话：用户在打招呼、感谢、闲聊，不涉及具体业务问题。

示例：“你好”“谢谢”“你是AI吗”“今天心情不好”

4. **clarification** - 引导澄清：用户的问题太模糊，缺少关键信息，无法确定意图。

示例：“有什么推荐的”“怎么办”“帮我看看”“出问题了”

判断规则：

- 结合对话历史判断，相同的话在不同上下文中意图可能不同

- 如果用户的问题涉及“我的”“查一下”等个性化表述，通常是工具调用

- 如果问题在问通用的规则、政策、产品信息，通常是知识检索

- 只有在真的无法判断意图时才分类为 **clarification**

- 以 **JSON** 格式输出，格式为：{"intent": "分类结果", "confidence": 置信度}

- 不要输出 **JSON** 以外的任何内容

对话历史：

```
{history}
```

用户最新消息：{query}

几个设计要点：

1. **每种意图都有明确定义和示例** ：这是 Few-shot 的典型用法，给模型足够的参考。定义要写清楚边界——什么算知识检索，什么算工具调用，不能让模型猜

2. **强调结合对话历史判断** ：同样一句“能退吗”，在不同的对话上下文中意图不同。第一轮对话时大概率是问退货政策（知识检索），前面在聊某个具体订单时大概率是想发起退货（工具调用）

3. **输出JSON格式** ：方便程序解析。加 confidence 字段可以在置信度低时触发兜底策略

4. **clarification要谨慎使用** ：Prompt 里明确说“只有在真的无法判断时才分类为 clarification”，避免模型动不动就要求澄清

2.2 分类效果与边界情况

大模型分类的准确率通常在 90% 以上，但有一些边界情况需要注意：

边界情况	用户消息	可能的分类	正确处理方式
上下文相关	能退吗？（前面在聊某个订单）	工具调用	结合对话历史判断，这里是想退货，不是问退货政策
知识 vs 工具模糊	iPhone 16 Pro 有货吗？	取决于系统设计	如果你有实时库存 API，分为工具调用；如果没有，分为知识检索
确认回复	好的（前一轮助手问“确认退货吗”）	工具调用（确认操作）	结合上下文，这是对退货操作的确认
多意图	查一下退货政策，顺便帮我看看订单到哪了	知识检索 + 工具调用	可以取主要意图，或者拆分处理（参考 Query 改写的多意图拆分）

边界情况是必然存在的。不要追求 100% 的分类准确率，而是要做好兜底策略——分类错了不要出大问题就行。比如把闲聊误分为知识检索，最多是检索了一下但没找到相关内容，模型给一个兜底回复，不会造成严重后果。但如果把工具调用误分为闲聊，用户想退货你回了个“好的呢”，那就有问题了。

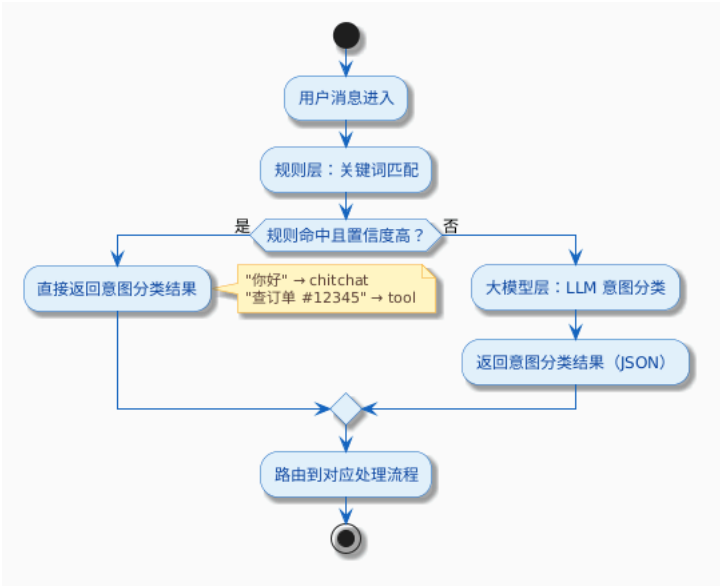
大模型分类方案的优缺点：

维度	表现
速度	较慢（500~3000ms）
准确率	高——能理解语义，能利用对话历史
成本	每次分类消耗一次模型 API 调用（用小模型可以控制成本）
维护成本	低——加新意图只需修改 Prompt，不需要写代码

3. 混合方案（推荐）

规则匹配速度快但准确率低，大模型分类准确但有延迟和成本。把两者结合起来，就是混合方案：**规则做第一层快速过滤，大模型做第二层精确分类**。

流程是这样的：



规则层只处理**高置信度** 的明确场景：

- 短消息且完全匹配闲聊关键词（“你好”“谢谢”）→ 直接判定为闲聊
- 包含明确的工具触发词且带具体参数（“查订单 #12345”）→ 直接判定为工具调用
- 其他所有情况 → 交给大模型分类

这样做的好处是：

- 大部分闲聊消息不需要调模型**：用户打招呼、说谢谢这种高频场景，规则就能搞定，省了一次 API 调用
- 复杂场景有模型兜底**：语义模糊、上下文相关的消息，交给大模型判断，准确率有保障
- 成本可控**：实际场景中，闲聊和明确的工具调用大约占 30%~40% 的消息量，这部分都可以被规则层拦截

三种方案的对比：

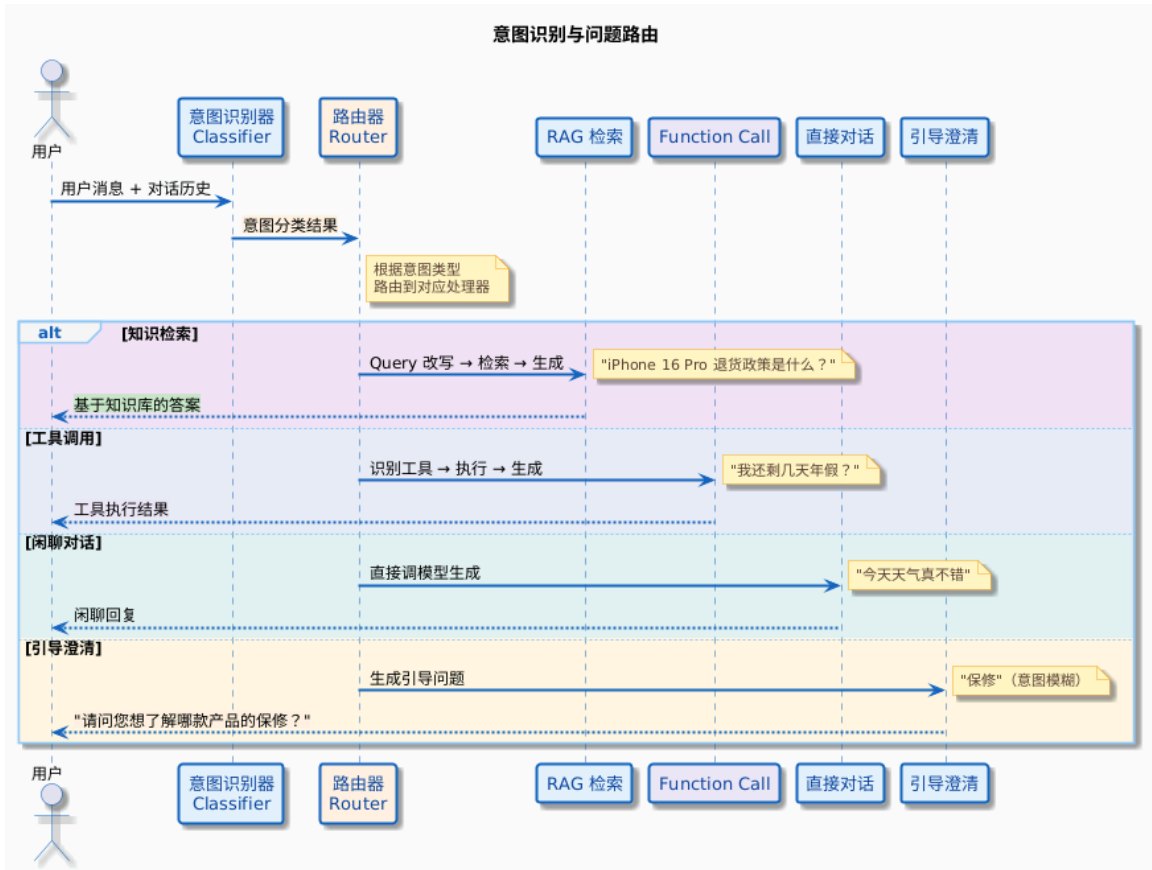
维度	规则匹配	大模型分类	混合方案
速度	极快（微秒）	较慢（300~3000ms）	快（规则命中时微秒，未命中时 300~3000ms）
准确率	低（60%~70%）	高（90%+）	高（90%+）
Token 成本	零	每次请求都消耗	仅未命中规则时消耗（节省 30%~40%）
维护成本	高（不断加规则）	低（改 Prompt）	中（规则 + Prompt 两套）
推荐场景	不推荐单独使用	消息量小、对延迟不敏感	生产环境（推荐）

实际项目中，混合方案是最稳的选择。规则层用来拦截最明确的意图，大模型层处理剩下的复杂场景。随着业务发展，你可以逐步丰富规则层的覆盖范围，减少大模型调用的比例。

问题路由的架构设计

1. 路由器模式

意图识别完成后，需要一个路由器把请求分发到不同的处理流程。整体架构是这样的：



路由器的职责很简单：拿到意图分类结果，调用对应的处理器。每个处理器各自负责自己的逻辑，互不干扰。

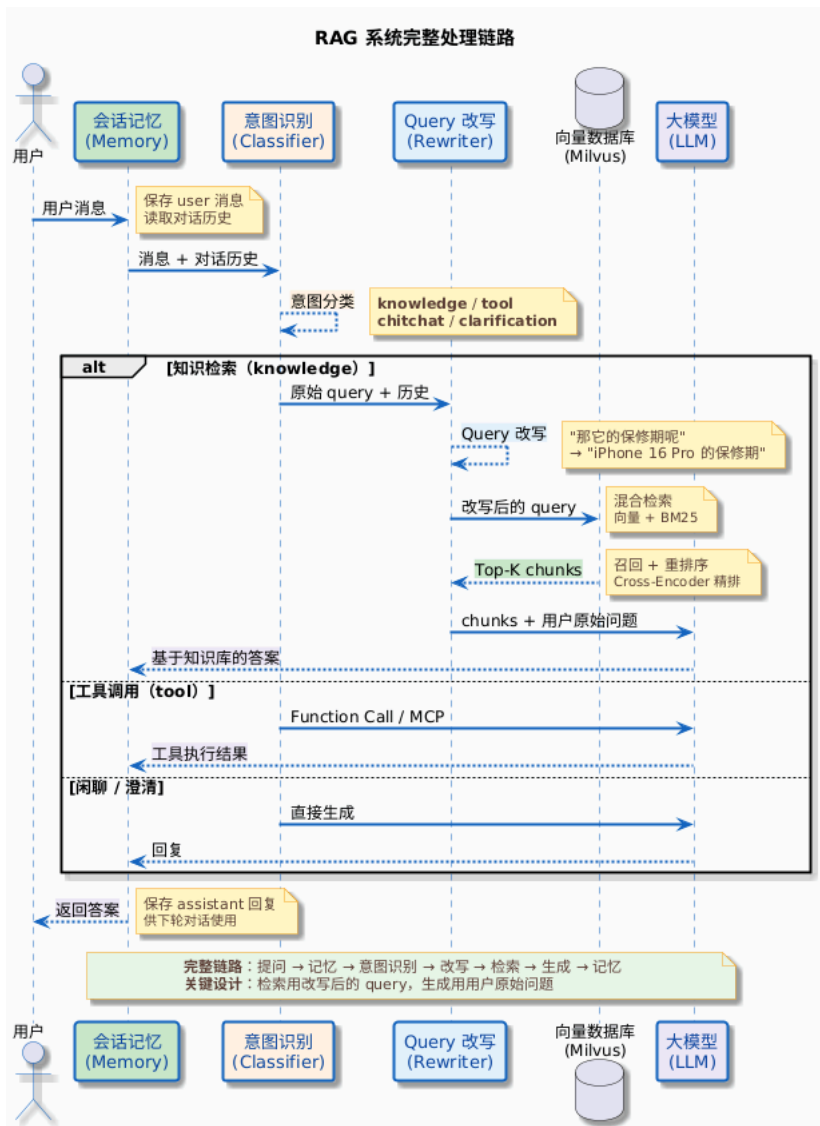
用 Java 代码表示，路由器的核心逻辑就是一个 switch：

```
public String route(String intent, List<Message> history, String query) throws IOException {
    switch (intent) {
        case "knowledge":
            // 知识检索路径: Query 改写 → 检索 → 生成答案
            return handleKnowledgeRetrieval(history, query);
        case "tool":
            // 工具调用路径: Function Call / MCP
            return handleToolInvocation(history, query);
        case "chitchat":
            // 闲聊路径: 直接调模型生成回答
            return handleChitchat(history, query);
        case "clarification":
            // 澄清路径: 返回引导问题
            return handleClarification(history, query);
        default:
            // 兜底: 默认走知识检索
            return handleKnowledgeRetrieval(history, query);
    }
}
```

default 分支走知识检索是一个安全的兜底策略。知识检索路径本身就有兜底机制——如果检索不到相关内容，模型会说“抱歉，找不到相关信息”。这比走错其他路径（比如误调了工具）要安全得多。

2. 意图识别在 RAG 完整链路中的位置

把意图识别加入后，完整的 RAG 链路变成了这样：



注意意图识别的位置：**在会话记忆之后，Query改写之前**。

为什么在会话记忆之后？因为意图识别可能需要对话历史来做判断。用户说“好的，帮我退了吧”，如果不知道前面在聊什么，无法判断这是工具调用（确认退货操作）。

为什么在Query改写之前？因为Query改写只在知识检索路径上才需要。如果意图是闲聊或工具调用，根本不需要做Query改写，直接走对应的路径就行。

不过在实际 Ragent 落地过程中，还是选择了问题重写在前，因为生产环境中，绝大多数还是基于知识库问答进行。

Java 实战：意图识别与路由的实现

1. 大模型意图分类器

下面是一个完整可运行的意图分类器，基于 Java + OkHttp 调用 SiliconFlow API：

```
public class IntentClassifier {

    private static final String API_URL = "https://api.siliconflow.cn/v1/chat/completions";
    private static final String API_KEY = "your api key";
    // 意图分类用小模型就够了
    private static final String MODEL = "Qwen/Qwen2.5-7B-Instruct";
    private static final OkHttpClient client = new OkHttpClient();
    private static final Gson gson = new Gson();

    /**
     * 意图分类 Prompt 模板
     */
    private static final String CLASSIFY_PROMPT = ""

    你是一个意图分类助手。根据对话历史和用户的最新消息，判断用户的意图类别。
```

意图类别定义：

1. **knowledge** - 知识检索：用户在询问产品信息、政策规定、操作指南等通用知识。
示例："iPhone 16 Pro 的退货政策是什么""保修期多久""配送范围覆盖哪些城市"
2. **tool** - 工具调用：用户想查询个人数据、实时信息，或执行某个操作。
示例："查一下我的订单状态""帮我申请退货""我还剩几天年假"
3. **chitchat** - 闲聊对话：用户在打招呼、感谢、闲聊，不涉及具体业务问题。
示例："你好""谢谢""你是AI吗""今天心情不好"
4. **clarification** - 引导澄清：用户的问题太模糊，缺少关键信息，无法确定意图。
示例："有什么推荐的""怎么办""帮我看看"

判断规则：

- 结合对话历史判断，相同的话在不同上下文中意图可能不同
- 如果用户的问题涉及"我的""查一下"等个人化表述，通常是工具调用
- 如果问题在问通用的规则、政策、产品信息，通常是知识检索
- 只有在真的无法判断意图时才分类为 **clarification**
- 以 **JSON** 格式输出：**{ "intent": "分类结果", "confidence": 置信度 }**
- 不要输出 **JSON** 以外的任何内容

对话历史：

%s

用户最新消息：%s"";

```
/**
 * 闲聊关键词（规则层快速过滤）
 */
private static final Set<String> CHITCHAT_KEYWORDS = Set.of(
    "你好", "您好", "谢谢", "感谢", "再见", "拜拜",
    "哈哈", "嗯嗯", "好的", "收到", "明白了", "ok"
);

/**
 * 混合方案：规则优先 + 大模型兜底
 */
public static IntentResult classify(List<Message> history,
                                   String query) throws IOException {
    // 第一层：规则快速过滤
    if (query.length() <= 6 && CHITCHAT_KEYWORDS.contains(query)) {
        return new IntentResult("chitchat", 0.99);
    }

    // 第二层：大模型分类
    return classifyByLLM(history, query);
}

/**
 * 调用大模型做意图分类
 */
private static IntentResult classifyByLLM(List<Message> history,
                                           String query) throws IOException {
    // 构建对话历史文本
    StringBuilder historyText = new StringBuilder();
    if (history.isEmpty()) {
        historyText.append("（无历史对话）");
    } else {
        for (Message msg : history) {
            String roleName = "user".equals(msg.role) ? "用户" : "助手";
            historyText.append(roleName).append(": ")
                .append(msg.content).append("\n");
        }
    }

    // 构建分类请求
    String prompt = String.format(CLASSIFY_PROMPT,
```



```

        historyText.toString(), query);

JsonObject body = new JsonObject();
body.addProperty("model", MODEL);
body.addProperty("temperature", 0.1);
body.addProperty("max_tokens", 100);
JsonArray messages = new JsonArray();
JsonObject userMsg = new JsonObject();
userMsg.addProperty("role", "user");
userMsg.addProperty("content", prompt);
messages.add(userMsg);
body.add("messages", messages);

Request request = new Request.Builder()
    .url(API_URL)
    .addHeader("Authorization", "Bearer " + API_KEY)
    .addHeader("Content-Type", "application/json")
    .post(RequestBody.create(body.toString(),
        MediaType.parse("application/json")))
    .build();

try (Response response = client.newCall(request).execute()) {
    assert response.body() != null;
    String responseBody = response.body().string();
    JsonObject json = gson.fromJson(responseBody, JsonObject.class);
    String content = json.getAsJsonArray("choices")
        .get(0).getAsJsonObject()
        .getAsJsonObject("message")
        .get("content").getString().trim();

    // 解析 JSON 结果
    return parseIntentResult(content);
}

/**
 * 解析模型返回的意图分类 JSON
 */
private static IntentResult parseIntentResult(String content) {
    try {
        JsonObject result = gson.fromJson(content, JsonObject.class);
        String intent = result.get("intent").getString();
        double confidence = result.has("confidence")
            ? result.get("confidence").getAsDouble() : 0.8;

        // 校验意图是否合法
        Set<String> validIntents = Set.of(
            "knowledge", "tool", "chitchat", "clarification");
        if (!validIntents.contains(intent)) {
            return new IntentResult("knowledge", 0.5);
        }

        return new IntentResult(intent, confidence);
    } catch (Exception e) {
        // JSON 解析失败，兜底走知识检索
        return new IntentResult("knowledge", 0.5);
    }
}

/**
 * 意图分类结果
 */
public static class IntentResult {
    public String intent;

```

```
public double confidence;

public IntentResult(String intent, double confidence) {
    this.intent = intent;
    this.confidence = confidence;
}

@Override
public String toString() {
    return String.format("{intent: \"%s\", confidence: %.2f}",
        intent, confidence);
}
}

/**
 * 消息数据结构
 */
public static class Message {
    public String role;
    public String content;

    public Message(String role, String content) {
        this.role = role;
        this.content = content;
    }
}
}
```

几个关键设计点:

混合方案 : `classify` 方法先用规则过滤短消息中的闲聊, 命中了就不调模型, 直接返回。没命中才调大模型

JSON解析兜底 : 模型偶尔会输出格式不规范的 JSON, `parseIntentResult` 方法做了异常处理, 解析失败时默认走知识检索

意图合法性校验 : 如果模型返回了未定义的意图类别 (比如自己编了个 “complaint”), 也兜底走知识检索

2. 路由器实现

路由器根据意图分类结果, 把请求分发到不同的处理逻辑:

```
public class IntentRouter {

    /**
     * 路由到对应的处理流程
     */
    public static String route(IntentClassifier.IntentResult intentResult,
        List<IntentClassifier.Message> history,
        String query) throws IOException {
        return switch (intentResult.intent) {
            case "tool" -> handleTool(history, query);
            case "chitchat" -> handleChitchat(query);
            case "clarification" -> handleClarification(query);
            default -> handleKnowledge(history, query);
        };
    }

    /**
     * 知识检索路径
     * 实际项目中: Query 改写 → 向量检索 → 重排序 → 生成答案
     */
    private static String handleKnowledge(List<IntentClassifier.Message> history,
        String query) {
        // 这里简化处理, 实际项目中要走完整的 RAG 流程
        return "[知识检索] 正在从知识库检索「" + query + "」相关内容并生成答案...";
    }
}
```

```
}

/**
 * 工具调用路径
 * 实际项目中：识别工具 → 传入参数 → 执行 → 返回结果
 */
private static String handleTool(List<IntentClassifier.Message> history,
                                String query) {
    // 这里简化处理，实际项目中要走 Function Call / MCP 流程
    return "[工具调用] 正在调用相关工具处理「" + query + "」...";
}

/**
 * 闲聊路径
 * 直接调模型生成回复，不带检索上下文
 */
private static String handleChitchat(String query) {
    // 简单的闲聊回复映射，实际项目中可以调模型生成
    if (query.contains("你好") || query.contains("您好")) {
        return "[闲聊] 您好！请问有什么可以帮您的？";
    }
    if (query.contains("谢谢") || query.contains("感谢")) {
        return "[闲聊] 不客气，还有其他问题随时问我。";
    }
    return "[闲聊] 好的，如果您有任何产品或服务相关的问题，随时告诉我。";
}

/**
 * 引导澄清路径
 * 返回引导性问题，让用户补充信息
 */
private static String handleClarification(String query) {
    return "[引导澄清] 您的问题我还不太明确，"
        + "能否告诉我您想了解哪方面的信息？"
        + "比如：产品信息、订单查询、退换货政策等。";
}
}
```

3. 多场景效果展示

把分类器和路由器串起来，跑几个典型场景看看效果：

```
public static void main(String[] args) throws IOException {
    // ===== 场景 1: 知识检索 =====
    System.out.println("===== 场景 1: 知识检索 =====");
    List<Message> history1 = List.of();
    String query1 = "iPhone 16 Pro 的退货政策是什么？";
    IntentResult result1 = IntentClassifier.classify(history1, query1);
    System.out.println("用户消息: " + query1);
    System.out.println("意图分类: " + result1);
    System.out.println("路由结果: " + IntentRouter.route(result1, history1, query1));

    // ===== 场景 2: 工具调用 =====
    System.out.println("\n===== 场景 2: 工具调用 =====");
    List<Message> history2 = List.of();
    String query2 = "帮我查一下订单 #12345 的物流状态";
    IntentResult result2 = IntentClassifier.classify(history2, query2);
    System.out.println("用户消息: " + query2);
    System.out.println("意图分类: " + result2);
    System.out.println("路由结果: " + IntentRouter.route(result2, history2, query2));

    // ===== 场景 3: 闲聊（规则层命中） =====
```

```
System.out.println("\n===== 场景 3: 闲聊（规则命中） =====");
List<Message> history3 = List.of();
String query3 = "你好";
IntentResult result3 = IntentClassifier.classify(history3, query3);
System.out.println("用户消息: " + query3);
System.out.println("意图分类: " + result3);
System.out.println("路由结果: " + IntentRouter.route(result3, history3, query3));

// ===== 场景 4: 引导澄清 =====
System.out.println("\n===== 场景 4: 引导澄清 =====");
List<Message> history4 = List.of();
String query4 = "有什么推荐的吗? ";
IntentResult result4 = IntentClassifier.classify(history4, query4);
System.out.println("用户消息: " + query4);
System.out.println("意图分类: " + result4);
System.out.println("路由结果: " + IntentRouter.route(result4, history4, query4));

// ===== 场景 5: 上下文相关的意图判断 =====
System.out.println("\n===== 场景 5: 上下文相关 =====");
List<Message> history5 = List.of(
    new Message("user", "iPhone 16 Pro 的退货政策是什么? "),
    new Message("assistant",
        "iPhone 16 Pro 支持七天无理由退货, 拆封后如有质量问题可联系售后。"),
    new Message("user", "我买的那台屏幕有亮点, 想退货"),
    new Message("assistant",
        "屏幕亮点属于质量问题, 可以申请退货。请问您需要我帮您发起退货申请吗? ")
);
String query5 = "好的, 帮我退了吧";
IntentResult result5 = IntentClassifier.classify(history5, query5);
System.out.println("对话历史: (用户在聊 iPhone 16 Pro 退货, 助手问是否发起退货申请)");
System.out.println("用户消息: " + query5);
System.out.println("意图分类: " + result5);
System.out.println("路由结果: " + IntentRouter.route(result5, history5, query5));
}
```

运行结果 (示意) :

```
===== 场景 1: 知识检索 =====
用户消息: iPhone 16 Pro 的退货政策是什么?
意图分类: {intent: "knowledge", confidence: 0.95}
路由结果: [知识检索] 正在从知识库检索「iPhone 16 Pro 的退货政策是什么?」相关内容并生成答案...

===== 场景 2: 工具调用 =====
用户消息: 帮我查一下订单 #12345 的物流状态
意图分类: {intent: "tool", confidence: 0.95}
路由结果: [工具调用] 正在调用相关工具处理「帮我查一下订单 #12345 的物流状态」...

===== 场景 3: 闲聊（规则命中） =====
用户消息: 你好
意图分类: {intent: "chitchat", confidence: 0.99}
路由结果: [闲聊] 您好! 请问有什么可以帮您的?

===== 场景 4: 引导澄清 =====
用户消息: 有什么推荐的吗?
意图分类: {intent: "clarification", confidence: 0.90}
路由结果: [引导澄清] 您的问题我还不太明确, 能否告诉我您想了解哪方面的信息? 比如: 产品信息、订单查询、退换货政策等。

===== 场景 5: 上下文相关 =====
对话历史: (用户在聊 iPhone 16 Pro 退货, 助手问是否发起退货申请)
用户消息: 好的, 帮我退了吧
意图分类: {intent: "tool", confidence: 0.95}
路由结果: [工具调用] 正在调用相关工具处理「好的, 帮我退了吧」...
```

几个关键观察：

- 场景1** ：典型的知识检索问题，问的是产品退货政策，应该去知识库里找答案
- 场景2** ：包含“查订单”“物流状态”等个人化表述，正确识别为工具调用
- 场景3** ：“你好”被规则层直接命中，没有调用大模型，响应更快
- 场景4** ：“有什么推荐的吗”太模糊，正确识别为需要澄清
- 场景5** ：这个最有意思——“好的，帮我退了吧”这句话单独看可能是闲聊（“好的”）或模糊表述，但结合对话历史（前面在聊退货，助手刚问了“要不要发起退货申请”），模型正确判断为工具调用。这就是对话历史在意图识别中的价值

生产环境的注意事项

1. 意图分类的准确率监控

意图分类的错误通常不会被用户直接感知到——用户只知道系统回答得不好，不知道是因为意图分错了导致走了错误的处理路径。所以需要主动监控。
建议记录每次分类的完整信息：

```
{
  "session_id": "session-001",
  "query": "好的，帮我退了吧",
  "intent": "tool",
  "confidence": 0.92,
  "method": "llm",
  "history_length": 4,
  "latency_ms": 280,
  "timestamp": "2025-03-07T14:30:00Z"
}
```

定期抽检分类日志，关注几个指标：

- 分类准确率** ：人工标注后计算，目标 90% 以上
- 各意图的分布比例** ：如果某类意图占比异常（比如 clarification 占了 30%），可能是 Prompt 太保守，动不动就要求澄清
- 误分类的典型案例** ：哪些消息被分错了？错分为什么类别？原因是什么？针对性调整 Prompt 或补充规则

规则层命中率：规则层拦截了多少比例的请求？如果太低（不到 10%），说明规则太少；如果太高（超过 60%），检查一下规则是否太激进导致误分

2. 意图分类失败的兜底

大模型返回结果可能出现几种异常情况：

- 返回了格式不规范的 JSON（比如多了个换行或注释）
- 返回了未定义的意图类别（比如模型自己编了个 complaint）
- API 调用超时或失败

这些情况都需要兜底。前面代码里已经做了处理——解析失败或意图不合法时，默认走知识检索。

```
public IntentResult safeClassify(List<Message> history, String query) {
    try {
        IntentResult result = classify(history, query);
        // 置信度太低也走兜底
        if (result.confidence < 0.5) {
            return new IntentResult("knowledge", 0.5);
        }
        return result;
    } catch (Exception e) {
        log.warn("意图分类失败，默认走知识检索: {}", e.getMessage());
        return new IntentResult("knowledge", 0.5);
    }
}
```

为什么兜底走知识检索而不是引导澄清？因为知识检索路径自带兜底——如果检索不到相关内容，生成策略里有兜底回复（“抱歉，未找到相关信息，建议您联系人工客服”）。而引导澄清会让用户觉得系统“太笨了，什么都要问”，体验更差。

3. 意图边界的动态调整

业务发展过程中，意图分类体系可能需要扩展。比如：

- 初期：knowledge / tool / chitchat / clarification 四类
- 中期：加入 complaint（投诉）、urgent（催单）等细分类别
- 后期：不同产品线可能有不同的意图体系

建议把意图分类体系做成**配置化**的：

```
// 意图定义配置（可以放在数据库或配置文件中）
List<IntentDefinition> intents = List.of(
    new IntentDefinition("knowledge", "知识检索",
        "用户在询问产品信息、政策规定、操作指南等通用知识",
        List.of("退货政策是什么", "保修期多久")),
    new IntentDefinition("tool", "工具调用",
        "用户想查询个人数据、实时信息，或执行某个操作",
        List.of("查我的订单", "帮我申请退货")),
    // 新增意图只需加配置，不改代码
    new IntentDefinition("complaint", "投诉",
        "用户在表达不满或投诉",
        List.of("我要投诉", "态度太差了"))
);
```

意图定义配置化之后，加新意图只需要修改配置，不需要改代码，也不需要重新部署。Prompt 模板从配置中动态生成，规则层的关键词也从配置中加载。

小结与下一篇预告

这篇讲了意图识别与问题路由，核心要点回顾：

1. **“一根筋”的RAG系统有问题**：所有消息都走 RAG 检索，闲聊消息检索到不相关内容反而更差，工具调用消息在知识库找不到答案
2. **四种意图分类**：知识检索（查知识库）、工具调用（调 API）、闲聊对话（直接回答）、引导澄清（反问用户）
3. **三种实现方案**：规则匹配（快但准确率低）、大模型分类（准但有延迟）、混合方案（推荐，规则做快速过滤 + 大模型做精确分类）
4. **意图识别在RAG链路中的位置**：会话记忆之后、Query 改写之前。意图识别需要对话历史作为输入，Query 改写只在知识检索路径上才需要
5. **兜底策略**：分类失败时默认走知识检索，这是最安全的路径

结合前面所有篇章，到这里多轮对话 RAG 系统的完整链路已经很清晰了：

用户提问→会话记忆（读取/保存对话历史）→意图识别（判断走哪条路）→路由分发：

知识检索路径：Query 改写 → 向量检索 + 重排序 → 生成答案

工具调用路径：Function Call / MCP → 模型生成最终答案

闲聊路径：模型直接生成回复

澄清路径：返回引导问题

→**保存assistant回复到会话记忆**

从数据入库到多轮对话，从检索生成到工具调用，再到意图识别和路由，RAG 系统的各个环节已经全部讲完了。但有一个关键问题还没有回答：**你怎么知道你的RAG系统好不好？**

检索召回的 chunk 是不是用户想要的？模型生成的答案是不是忠实于检索到的内容？端到端的用户满意度怎么样？哪个环节是瓶颈？

靠人工抽检能发现一些问题，但效率太低，覆盖不全。你需要一套系统化的评估方法，用数据来量化 RAG 系统每个环节的效果，找到短板，驱动优化。

下一篇咱们来聊 **RAG系统的评估与优化** ——怎么量化评估检索准确率、生成忠实度、端到端满意度，怎么通过数据驱动持续优化 RAG 系统的各个环节。

